

WEEK 2 REPORT

Product and Category Management System

Objective

The objective of this task is to create a database system for managing products and their categories in an e-commerce company.

The system stores details about categories and products. It also connects products with their respective categories and allows us to perform different SQL operations on the data.

2. Scenario

An e-commerce company needs to maintain information about its products and categories.

Each product belongs to a particular category. The database stores important details such as:

- Product ID
- Product Name
- Price
- Stock
- Category ID

Two tables are created to manage this information:

1. Category Table
2. Product Table

3. Creating the Tables

Category Table:

The Category table is used to store category details.

```
SQL> CREATE TABLE CATEGORY(CATEGORY_ID NUMBER PRIMARY KEY,CATEGORY_NAME VARCHAR2(50),DESCRIPTION VARCHAR2(200));  
Table created.
```

Product Table:

The Product table is used to store product details.

```
SQL> CREATE TABLE PRODUCT (PRODUCT_ID VARCHAR2(7),PRODUCT_NAME VARCHAR(20),CATEGORY_ID VARCHAR2(7),PRICE NUMBER(9,2),STOCK_QUANTITY NUMBER);  
Table created.
```

4. Defining Relationship and Creating Constraints

The relationship between the Category and Product tables is **One-to-Many (1)**.

This means **one category can have many products**.

For example, the Electronics category can contain products such as laptops, headphones, and mobile phones.

The **Category_ID** is used to connect the two tables.

Primary Key:

A primary key is used to identify each record uniquely in a table.

In our database:

→ CATEGORY_ID is the Primary Key of the CATEGORY table.

→ PRODUCT_ID is used to identify each product uniquely.

The CATEGORY_ID values should not be repeated in the Category table.

```
SQL> ALTER TABLE PRODUCT ADD CONSTRAINT PK_PRODUCT PRIMARY KEY (PRODUCT_ID);  
Table altered.
```

Creating Relationship using FOREIGN KEY:

The CATEGORY_ID from the Product table is connected to the CATEGORY_ID in the Category table.

```
SQL> ALTER TABLE PRODUCT  
2  ADD CONSTRAINT FK_PRODUCT_CATEGORY  
3  FOREIGN KEY (CATEGORY_ID)  
4  REFERENCES CATEGORY(CATEGORY_ID);  
  
Table altered.
```

This prevents a product from being assigned to a category that does not exist.

ADDING CONSTRAINTS:

NOT NULL:

The NOT NULL constraint makes sure that important fields are not left empty.

```
SQL> ALTER TABLE CATEGORY
  2  MODIFY CATEGORY_NAME CONSTRAINT NN_CATEGORY_NAME NOT NULL;
```

Table altered.

```
SQL> ALTER TABLE PRODUCT
  2  MODIFY PRODUCT_NAME CONSTRAINT NN_PRODUCT_NAME NOT NULL;
```

Table altered.

```
SQL> ALTER TABLE PRODUCT
  2  MODIFY PRICE CONSTRAINT NN_PRODUCT_PRICE NOT NULL;
```

Table altered.

CHECK Constraint

The CHECK constraint ensures that the value follows a given condition.

For example, the product price should be greater than zero.

```
SQL> ALTER TABLE PRODUCT
  2  ADD CONSTRAINT CHK_PRODUCT_PRICE
  3  CHECK (PRICE > 0);
```

Table altered.

```
SQL> ALTER TABLE PRODUCT
  2  ADD CONSTRAINT CHK_PRODUCT_STOCK
  3  CHECK (STOCK_QUANTITY >= 0);
```

Table altered.

UNIQUE Constraint

The UNIQUE constraint prevents duplicate values.

```
SQL> ALTER TABLE CATEGORY
  2  ADD CONSTRAINT UQ_CATEGORY_NAME
  3  UNIQUE (CATEGORY_NAME);

Table altered.
```

These constraints help to maintain accurate and valid data in the tables.

Inserting Values

After creating the tables and applying the constraints, values are inserted into the Category and Product tables.

```
SQL> INSERT INTO CATEGORY VALUES(&CATEGORY_ID,&CATEGORY_NAME,&DESCRIPTION');
Enter value for category_id: 1
Enter value for category_name: Electronics
Enter value for description: Electronic devices and accessories
old  1: INSERT INTO CATEGORY VALUES(&CATEGORY_ID,&CATEGORY_NAME,&DESCRIPTION')
new  1: INSERT INTO CATEGORY VALUES(1,'Electronics','Electronic devices and accessories')

1 row created.

SQL> /
Enter value for category_id: 2
Enter value for category_name: Clothing
Enter value for description: Men and women clothing products
old  1: INSERT INTO CATEGORY VALUES(&CATEGORY_ID,&CATEGORY_NAME,&DESCRIPTION')
new  1: INSERT INTO CATEGORY VALUES(2,'Clothing','Men and women clothing products')

1 row created.
```

CATEGORY TABLE:

```
SQL> SELECT * FROM CATEGORY;
```

CATEGORY_ID	CATEGORY_NAME	DESCRIPTION
1	Electronics	Electronic devices and accessories
2	Clothing	Men and women clothing products
3	Books	Educational and general books
4	Home Appliances	Appliances used at home
5	Sports	Sports and fitness equipment
6	Beauty	Beauty and personal care products
7	Grocery	Daily grocery and food products
8	Footwear	Shoes and other footwear
9	Furniture	Home and office furniture
10	Toys	Toys and games for children

10 rows selected.

PRODUCT TABLE:

```
SQL> INSERT INTO PRODUCT VALUES('&PRODUCT_ID','&PRODUCT_NAME',&CATEGORY_ID,&PRICE,&STOCK_QUANTITY);
Enter value for product_id: P001
Enter value for product_name: Smartphone
Enter value for category_id: 1
Enter value for price: 24999.00
Enter value for stock_quantity: 50
old 1: INSERT INTO PRODUCT VALUES('&PRODUCT_ID','&PRODUCT_NAME',&CATEGORY_ID,&PRICE,&STOCK_QUANTITY)
new 1: INSERT INTO PRODUCT VALUES('P001','Smartphone',1,24999.00,50)

1 row created.

SQL> /
Enter value for product_id: P002
Enter value for product_name: Wireless Mouse
Enter value for category_id: 1
Enter value for price: 799.00
Enter value for stock_quantity: 100
old 1: INSERT INTO PRODUCT VALUES('&PRODUCT_ID','&PRODUCT_NAME',&CATEGORY_ID,&PRICE,&STOCK_QUANTITY)
new 1: INSERT INTO PRODUCT VALUES('P002','Wireless Mouse',1,799.00,100)

1 row created.
```

```
SQL> SELECT * FROM PRODUCT;
```

PRODUCT	PRODUCT_NAME	CATEGORY_ID	PRICE	STOCK_QUANTITY
P001	Smartphone	1	24999	50
P002	Wireless Mouse	1	799	100
P003	Bluetooth Speaker	1	1499	75
P004	Cotton Shirt	2	899	60
P005	Denim Jeans	2	1599	45
P006	Data Structures	3	650	30
P007	Python Programming	3	850	25
P008	Electric Kettle	4	1299	35
P009	Running Shoes	5	2199	40
P010	Office Chair	9	5499	15
P011	Building Blocks	10	999	35
P012	Cooking Oil	7	699	70
P013	Face Wash	6	349	90
P014	Study Table	9	6999	10
P015	Yoga Mat	5	599	80

```
15 rows selected.
```

CRUD Operations

CRUD stands for Create, Read, Update, and Delete. These operations are used to manage the records in the database.

Insert Record:

A new product can be added using the INSERT command.

```
SQL> INSERT INTO PRODUCT VALUES('P016','Smart watch',1,2999.00,40);
```

```
1 row created.
```

Update Product Price:

The UPDATE command is used to change existing data.

```
SQL> UPDATE PRODUCT SET PRICE = '7999.00' WHERE PRODUCT_ID = 'P014';  
1 row updated.
```

Delete a Product:

The DELETE command is used to remove a product.

```
SQL> DELETE FROM PRODUCT WHERE PRODUCT_ID = 'P016';  
1 row deleted.
```

Category-wise Product Reports:

SQL queries can be used to generate different reports from the Product and Category tables.

Display All Products Under Each Category :

```
SQL> SELECT c.CATEGORY_NAME,  
2         p.PRODUCT_ID,  
3         p.PRODUCT_NAME,  
4         p.PRICE,  
5         p.STOCK_QUANTITY  
6 FROM CATEGORY c  
7 JOIN PRODUCT p  
8 ON c.CATEGORY_ID = p.CATEGORY_ID  
9 ORDER BY c.CATEGORY_NAME;
```

CATEGORY_NAME	PRODUCT	PRODUCT_NAME	PRICE	STOCK_QUANTITY
Beauty	P013	Face Wash	349	90
Books	P006	Data Structures	650	30
Books	P007	Python Programming	850	25
Clothing	P004	Cotton Shirt	899	60
Clothing	P005	Denim Jeans	1599	45
Electronics	P001	Smartphone	24999	50
Electronics	P002	Wireless Mouse	799	100
Electronics	P003	Bluetooth Speaker	1499	75
Furniture	P010	Office Chair	5499	15
Furniture	P014	Study Table	7999	10
Grocery	P012	Cooking Oil	699	70
Home Appliances	P008	Electric Kettle	1299	35
Sports	P009	Running Shoes	2199	40
Sports	P015	Yoga Mat	599	80
Toys	P011	Building Blocks	999	35

15 rows selected.

Count Number of Products in Each Category :

```
SQL> SELECT c.CATEGORY_NAME,  
2         COUNT(p.PRODUCT_ID) AS PRODUCT_COUNT  
3 FROM CATEGORY c  
4 LEFT JOIN PRODUCT p  
5 ON c.CATEGORY_ID = p.CATEGORY_ID  
6 GROUP BY c.CATEGORY_NAME  
7 ORDER BY c.CATEGORY_NAME;
```

CATEGORY_NAME	PRODUCT_COUNT
Beauty	1
Books	2
Clothing	2
Electronics	3
Footwear	0
Furniture	2
Grocery	1
Home Appliances	1
Sports	2
Toys	1

10 rows selected.

Find the Highest-Priced Product in Each Category :

```
SQL> SELECT c.CATEGORY_NAME,  
2         p.PRODUCT_NAME,  
3         p.PRICE  
4 FROM CATEGORY c  
5 JOIN PRODUCT p  
6 ON c.CATEGORY_ID = p.CATEGORY_ID  
7 WHERE p.PRICE = (  
8     SELECT MAX(p2.PRICE)  
9     FROM PRODUCT p2  
10    WHERE p2.CATEGORY_ID = p.CATEGORY_ID  
11 )  
12 ORDER BY c.CATEGORY_NAME;
```

CATEGORY_NAME	PRODUCT_NAME	PRICE
Beauty	Face Wash	349
Books	Python Programming	850
Clothing	Denim Jeans	1599
Electronics	Smartphone	24999
Furniture	Study Table	7999
Grocery	Cooking Oil	699
Home Appliances	Electric Kettle	1299
Sports	Running Shoes	2199
Toys	Building Blocks	999

9 rows selected.

Display Categories Having More Than 5 Products:

```
SQL> SELECT c.CATEGORY_NAME,  
2         COUNT(p.PRODUCT_ID) AS PRODUCT_COUNT  
3 FROM CATEGORY c  
4 JOIN PRODUCT p  
5 ON c.CATEGORY_ID = p.CATEGORY_ID  
6 GROUP BY c.CATEGORY_NAME  
7 HAVING COUNT(p.PRODUCT_ID) > 3  
8 ORDER BY PRODUCT_COUNT DESC;
```

no rows selected

```
SQL> SELECT c.CATEGORY_NAME,  
2         COUNT(p.PRODUCT_ID) AS PRODUCT_COUNT  
3 FROM CATEGORY c  
4 JOIN PRODUCT p  
5 ON c.CATEGORY_ID = p.CATEGORY_ID  
6 GROUP BY c.CATEGORY_NAME  
7 HAVING COUNT(p.PRODUCT_ID) > 2  
8 ORDER BY PRODUCT_COUNT DESC;
```

CATEGORY_NAME	PRODUCT_COUNT
Electronics	3

Find Average Product Price Category-wise:

```
SQL> SELECT c.CATEGORY_NAME,  
2         ROUND(AVG(p.PRICE), 2) AS AVERAGE_PRICE  
3 FROM CATEGORY c  
4 JOIN PRODUCT p  
5 ON c.CATEGORY_ID = p.CATEGORY_ID  
6 GROUP BY c.CATEGORY_NAME  
7 ORDER BY c.CATEGORY_NAME;
```

CATEGORY_NAME	AVERAGE_PRICE
Beauty	349
Books	750
Clothing	1249
Electronics	9099
Furniture	6749
Grocery	699
Home Appliances	1299
Sports	1399
Toys	999

9 rows selected.

Conclusion:

The Product and Category Management System was successfully created using SQL. Two tables, Category and Product, were designed and connected using a One-to-Many relationship. Primary Key and Foreign Key constraints were used to maintain the relationship between the tables, while NOT NULL, CHECK, and UNIQUE constraints helped maintain valid and consistent data. The database was tested by inserting records and performing CRUD operations such as inserting, displaying, updating, and deleting product details. Category-wise SQL queries were also used to generate useful information such as product counts, highest prices, and average prices. Overall, this task helped in understanding how tables, relationships, constraints, and SQL operations are used together to manage data efficiently.

